

MIDTERM EVALUATION CONTROLLER PROGRAMMING COURSE



Prepared by : Else Kusuma
: Selly Ajeng Via Wiranti
Student ID : 20242241020
: 2042241076
Class : Class 4C
Lecturer : Ahmad Radhy, S.Si., M.Si

Instrumentation Technology Engineering Study Program
Even Semester 2025/2026
May 3, 2026

TABEL OF CONTENTS

Table of Contents

Daftar Isi

1	TABEL OF CONTENTS	1
2	A. Systematic Literature Review	2
2.1	Integrated <i>Future Works</i>	5
3	B. Proposed Methodology	7
3.1	Block Diagram	7
3.2	Flowchart	8
3.3	Method Description	9
4	C. Simulation and Results	11
4.1	Tools and Software	11
4.2	Simulation Circuit	12
4.3	Data and GNU Plot	13
4.4	GNU Plot Result Analysis	13
5	D. Method Advantages	15
5.1	Comparison with Reference Journal Methods	15
5.2	Originality Contributions	15
6	E. GitHub and LinkedIn	17
7	Conclusion	18

A. Systematic Literature Review

Tabel 1: Summary of 15 Reference Journals

No	Article Title	Authors (Year)	Method	Main Results	Future Work
1	An Empirical Study on Bugs in Rust Programming Language	Sijie Yu & Ziyuan Wang (2024)	Mining 10,097 bug reports and 9,360 revisions from Rust GitHub repositories. Manual annotation of 194 sampled bugs. Pearson and Spearman correlation analysis.	Compiler bugs dominated (72.77%). Semantic bugs were the most common root cause (59.28%). ICE was the dominant symptom (83.87%). [1]	Develop automated bug detection tools and expand analysis to broader Rust ecosystems.
2	Learning and Programming Challenges of Rust: A Mixed-Methods Study	Zhu et al. (2022)	Empirical study of 100 Rust Stack Overflow questions and survey of 101 programmers using LDA topic modeling.	Lifetime computation is harder than ownership rules. Enhanced compiler error messages significantly improved understanding. [2]	Improve compiler error messages and develop automated IDE assistance tools.
3	Tiny Machine Learning for Resource-Constrained Microcontrollers	Immonen & Hämäläinen (2022)	Comprehensive literature review on TinyML optimization methods and frameworks.	8-bit quantization reduced memory usage by 75%. No unified TinyML framework exists for all hardware. [3]	Develop unified TinyML frameworks and open-source benchmarking datasets.

4	A System-Level Test Methodology for Communication Peripherals in System-on-Chips	Angione et al. (2025)	SLT methodology using ATPG, LBIST and MBIST analysis with FPGA-based error injection.	SAF coverage increased from 97.89% to 99.01%. TDF coverage improved by 1.51%. [4]	Apply the methodology to other peripherals such as UART and SPI.
5	Machine Learning for Microcontroller-Class Hardware: A Review	Saha et al. (2022)	Quantitative review of TinyML workflows and benchmarking across multiple applications.	Pruning and quantization achieved 16× model reduction. CMSIS-NN provided 4.6× speedup. [5]	Integrate neuro-symbolic reasoning and improve TinyML security mechanisms.
6	Fleet Management System for Electric Buses using ESP32 Microcontroller	Ganiss Roger L & P. Nisha (2024)	ESP32-based fleet monitoring system integrated with sensors and cloud dashboards.	Real-time monitoring of battery health, speed, vibration, and location was successfully implemented. [6]	Integrate AI/ML for predictive maintenance and smart-city transportation systems.
7	Basic Developing Environment of Microcontroller-based Monitoring System for Physiological Signals	Man Xie et al. (2022)	ESP32 monitoring system with physiological sensors and RNN-based signal denoising.	Breath-rate accuracy exceeded 90% with reduced communication data volume. [7]	Develop more advanced neural networks and perform clinical validation.
8	A wireless sensor network for IoT platform for consumption and quality monitoring of water	Christodoulos Axiotidis, Evangelia Konstantopoulou, Nicolas Slavkos (2025)	IoT implementation based on ESP32 with water quality sensors and wireless communication	The system successfully monitored water quality in real time and detected water contamination. [8]	Development of large-scale monitoring systems and improvement of sensor accuracy

9	Applying hypervisor-based fault tolerance techniques to safety-critical embedded systems	Santiago Lozano, Javier Fernandez, Jesus Carretero (2026)	Hypervisor-Based Fault Tolerance (HBFT) in embedded systems	The system successfully detected and recovered faults in real-time embedded systems. [9]	Optimization of fault tolerance mechanisms and resource efficiency
10	Automated tomato leaf disease detection and alert system using Internet of Things and TinyML	Timothy Malche, Mukesh Joshi, Govind Murari Upadhyay, Pramod Kumar Soni (2026)	TinyML and CNN implementation on ESP32-S3 with MQTT communication	The system achieved 94.6% disease detection accuracy with low power consumption. [10]	Development of lighter and faster TinyML models
11	Design and Development of Test Set-up with Integrated Digital Controller for Temperature Radiation of Materials	Anjan Dutta Gupta, Sabyasachi Pathak, Anirban De, Tanumoy Bhattacharya, Shankar Kumar, Mousumi Garai, Pranab Bhattacharya, Soumita Chakraborty, Argha Dutta, Santu Dey, N. Gayathri, Paramita Mukherjee (2026)	Development of a microcontroller-based digital temperature controller	The system maintained a temperature stability up to 500°C [11]	Development of more accurate temperature control systems

12	Framework and Tooling Proposals for Agile Certification of Safety-Critical Embedded Software in Avionics Systems	Claude Baron, Vincent Louis (2023)	Agile continuous certification for avionics embedded software	The framework improved the efficiency of avionics software certification processes. [12]	Development of the more adaptive automated certification tools
13	Layered Control Architectures for AI Safety: Cybersecurity-Oriented Systems Framework	Young B. Choi, Paul A. C. Hong, Young Soo Park (2026)	Layered control architecture based on cybersecurity principles	The framework improved AI system security and reliability. [13]	Validation of the framework in real-world systems
14	Programming Microcontrollers through High-Level Abstractions: The OMicroB Project	Steven Vauvrams, Basile Pessin, Benoît Vauçon, Emmanuel Chailloux (2023)	Microcontroller programming using an OCaml virtual machine	The approach provided safer and more efficient high-level programming abstractions. [14]	Development of compatibility with more microcontroller architectures
15	Safety and Security Priorities of Smart Embedded Devices in a Technology Enterprise	Negin Moghadasi, Zachary A. Collier, Davis C. Loose, Matthew C. Gunn, Megan E. Gunn, Igor Linkov, Karen R. Jackson, James H. Lambert (2026)	AI-based risk management framework for IoT embedded devices.	The system successfully prioritized security and operational risks in IoT systems. [15]	Development of more adaptive dynamic security systems

Integrated *Future Works*

FW1. Development of *automated bug detection tools* and expansion of analysis across broader Rust ecosystems to improve *embedded software* reliability.

- FW2.** Improvement of *compiler error messages* and development of *automated IDE assistance tools* to simplify Rust-based system development.
- FW3.** Development of *unified TinyML frameworks* and *open-source benchmarking datasets* for efficient cross-platform TinyML deployment.
- FW4.** Application of the SLT testing methodology to other peripherals such as UART and SPI to improve communication reliability in *embedded systems*.
- FW5.** Integration of *neuro-symbolic reasoning* and enhancement of TinyML security mechanisms to strengthen *edge AI* intelligence and protection.
- FW6.** Integration of AI and *machine learning* for *predictive maintenance* and the development of *smart-city* transportation systems.
- FW7.** Development of more advanced *neural networks* and clinical validation to improve physiological signal monitoring accuracy.
- FW8.** Development of large-scale monitoring systems and enhancement of sensor accuracy in IoT-based water quality monitoring platforms.
- FW9.** Optimization of *fault tolerance* mechanisms and resource efficiency in *safety-critical embedded systems*.
- FW10.** Development of lighter and faster TinyML models to improve *microcontroller* inference efficiency.
- FW11.** Development of higher-accuracy temperature control systems to support industrial stability and material experimentation.
- FW12.** Development of more adaptive *automated certification tools* to support *continuous certification* processes in avionics *embedded software*.
- FW13.** Validation of *layered control architectures* for AI *safety* implementation in real-world and *distributed systems* environments.
- FW14.** Expansion of OMicroB framework compatibility with more *microcontroller* architectures to improve *embedded programming* portability.
- FW15.** Development of more adaptive *dynamic security systems* to improve security and risk management in *smart embedded IoT devices*.

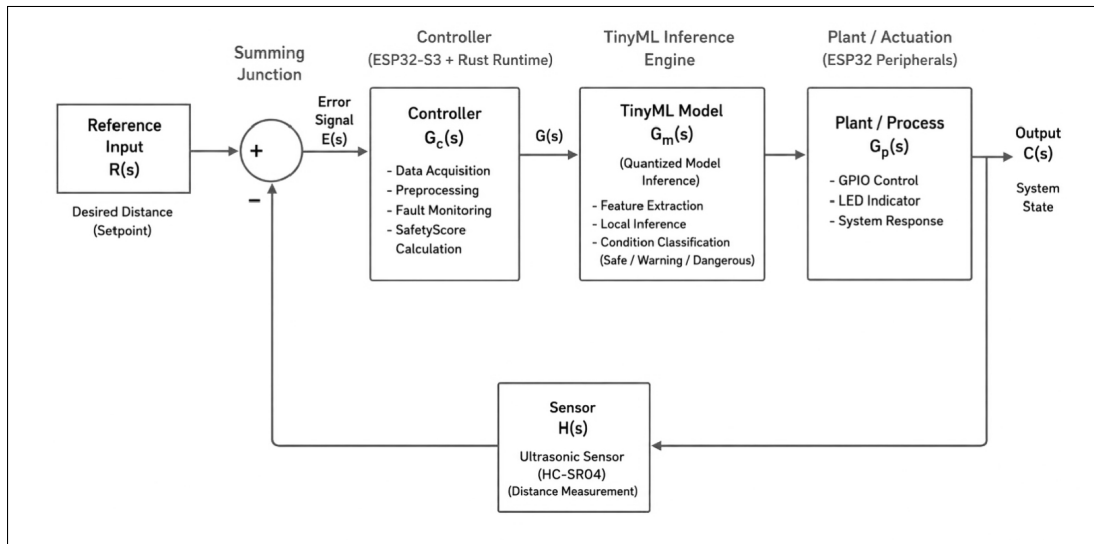
B. Proposed Methodology

Method Title

SAFE-TinyRust: Rust-Based Adaptive Distance Monitoring and Warning System using TinyML on ESP32-S3

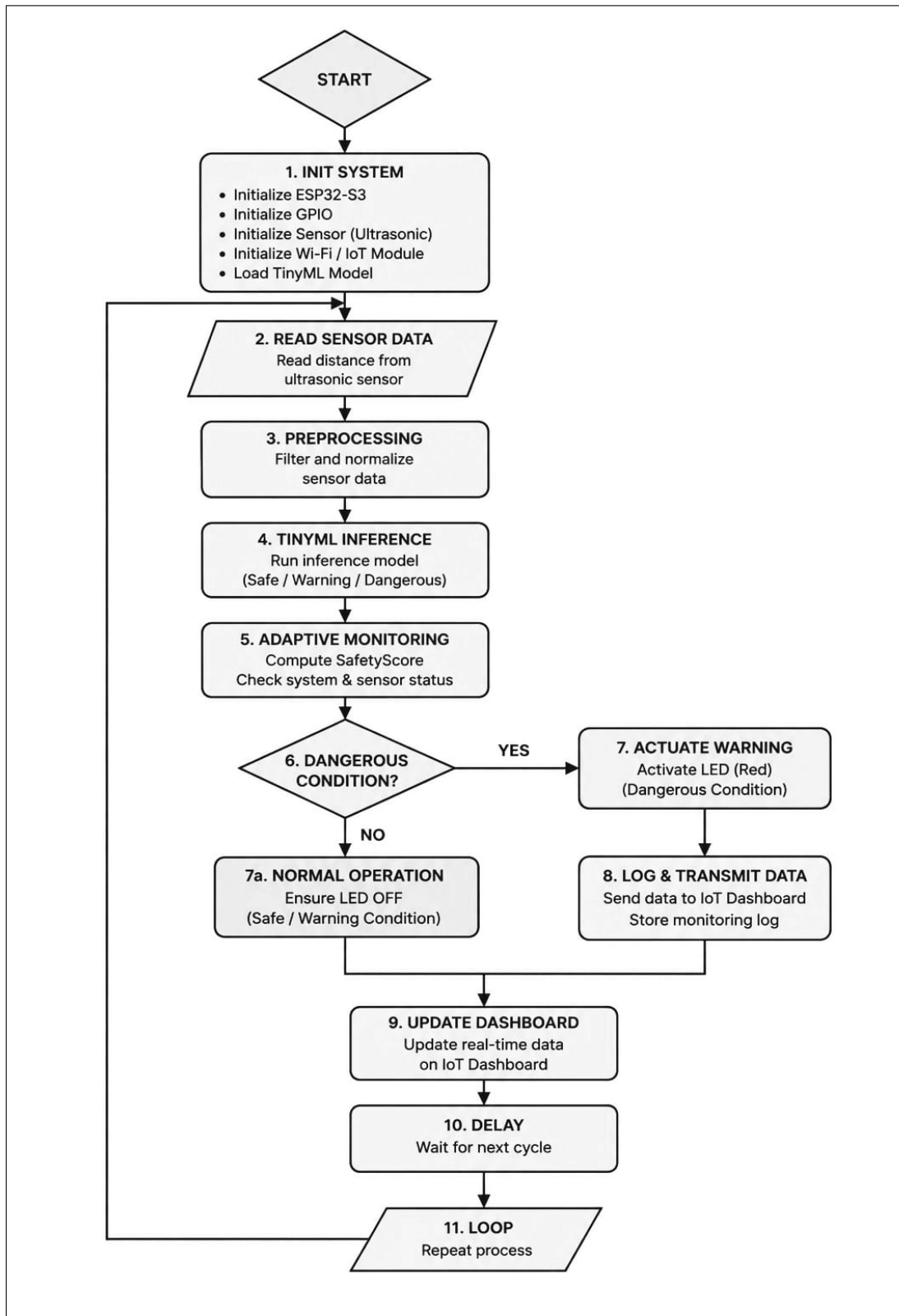
Rationale: The proposed SAFE-TinyRust framework is developed by integrating future work directions from the 15 reviewed journals, particularly in TinyML optimization, Rust-based embedded systems, adaptive monitoring, fault tolerance, IoT communication, and AI-based intelligent instrumentation. These integrated future works form the foundation of the SAFE-TinyRust framework for adaptive real-time distance monitoring and intelligent warning systems using ESP32-S3.

Block Diagram



Gambar 1: Block diagram of the proposed methodology.

Flowchart



Gambar 2: Flowchart of the proposed algorithm.

Method Description

The proposed method, called **SAFE-TinyRust**, integrates multiple future work concepts from previous studies into a unified approach for programming intelligent sensors and actuators on embedded IoT systems. The framework combines Rust-based embedded programming, TinyML inference, adaptive monitoring, fault-tolerant mechanisms, dynamic security systems, and IoT communication using the ESP32-S3 microcontroller platform.

The proposed system begins with sensor initialization and low-level hardware configuration on the ESP32-S3. Distance sensors continuously acquire real-time environmental data, which are processed using TinyML models deployed directly on the microcontroller. Unlike conventional embedded systems that rely heavily on cloud processing, the proposed method performs lightweight AI inference locally on the device to reduce latency, improve energy efficiency, and enhance real-time responsiveness.

This method integrates the future work of TinyML optimization, lightweight inference models, and intelligent monitoring systems by implementing adaptive distance classification and anomaly detection. The TinyML model analyzes sensor readings and determines whether the detected distance is categorized as safe, warning, or dangerous. If dangerous conditions are detected, the system automatically activates actuators such as LEDs as an early warning mechanism.

The proposed framework also integrates safety-critical and fault-tolerant concepts by implementing adaptive monitoring and runtime validation. Fault monitoring mechanisms continuously check sensor conditions, communication stability, and inference consistency to improve operational reliability. Dynamic security layers are also incorporated to protect the embedded system against abnormal behaviors and communication disturbances.

To improve software reliability and memory safety, the entire embedded application is developed using the Rust programming language. Rust provides ownership-based memory management and thread safety, reducing common embedded programming issues such as memory leaks, race conditions, and unsafe pointer access. The implementation uses Rust embedded crates such as `esp-idf-hal`, `embedded-hal`, and `tf-lite-micro-rs` to control GPIO, sensor communication, TinyML inference, and actuator management.

The GPIO registers of the ESP32-S3 are configured through Rust hardware abstraction layers (HAL). For example, digital outputs for LEDs and buzzers are initialized using GPIO output configurations, while ultrasonic sensor communication uses GPIO trigger and echo pin control. In conventional C/C++ programming, these operations are commonly implemented through direct register manipulation such as:

```
GPIO.enable_w1ts = (1 << LED_PIN);
```

```
GPIO.out_w1ts = (1 << LED_PIN);
```

However, in the proposed Rust-based framework, safer abstractions are used:

```
let mut led = PinDriver::output(peripherals.pins.gpio2)?;  
led.set_high()?;
```

This approach improves code readability, portability, and system safety while maintaining efficient low-level hardware control.

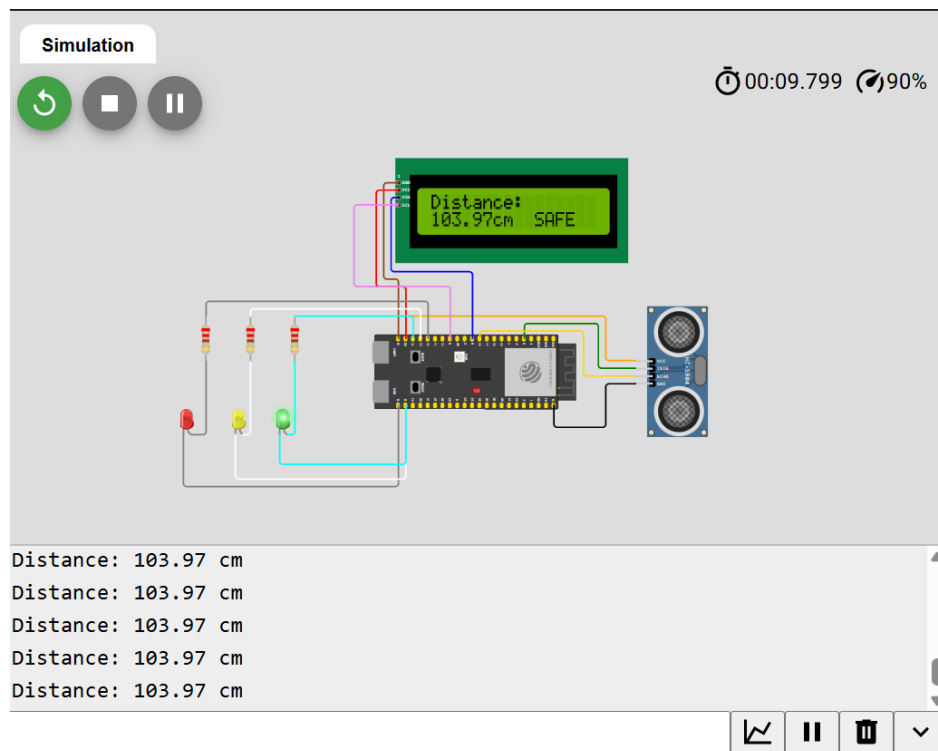
Finally, the monitoring results and warning status are transmitted to an IoT dashboard through wireless communication for remote monitoring and data visualization. By integrating TinyML, Rust safety mechanisms, fault tolerance, adaptive security, and intelligent actuator control, the proposed method produces a lightweight, secure, and intelligent embedded monitoring architecture suitable for modern IoT applications and safety-critical embedded systems.

C. Simulation and Results

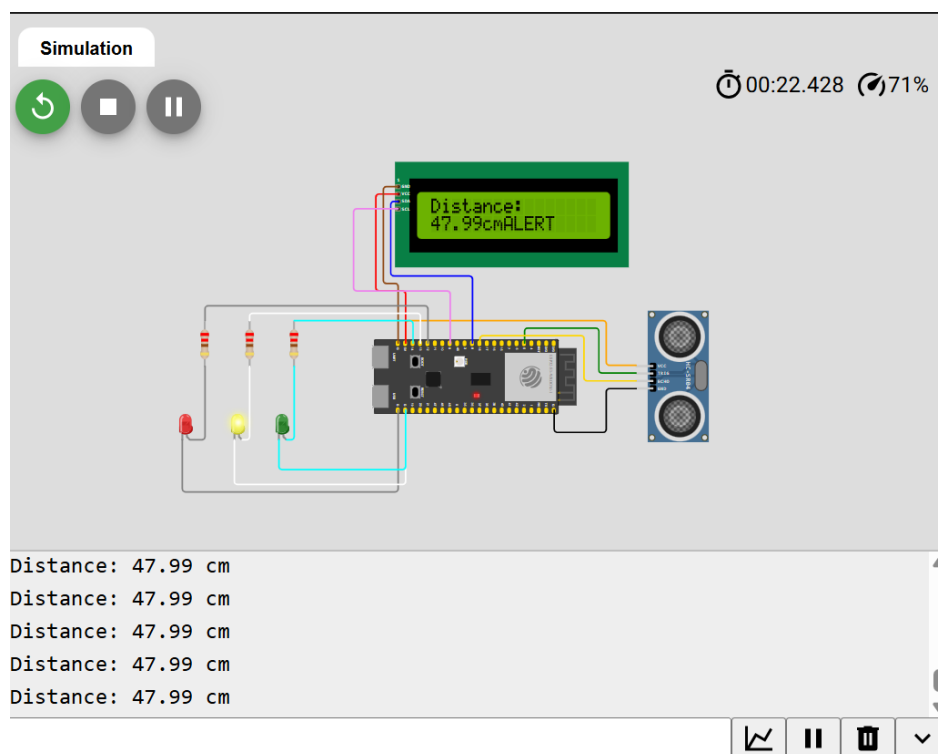
Tools and Software

- **Simulator:** Proteus 8.17
- **IDE:** Visual Studio Code with Rust Extension
- **Configuration:** ESP32-S3 Rust Environment
- **Plotting:** GNUPlot
- **Microncontroller:** ESP32-S3

Simulation Circuit

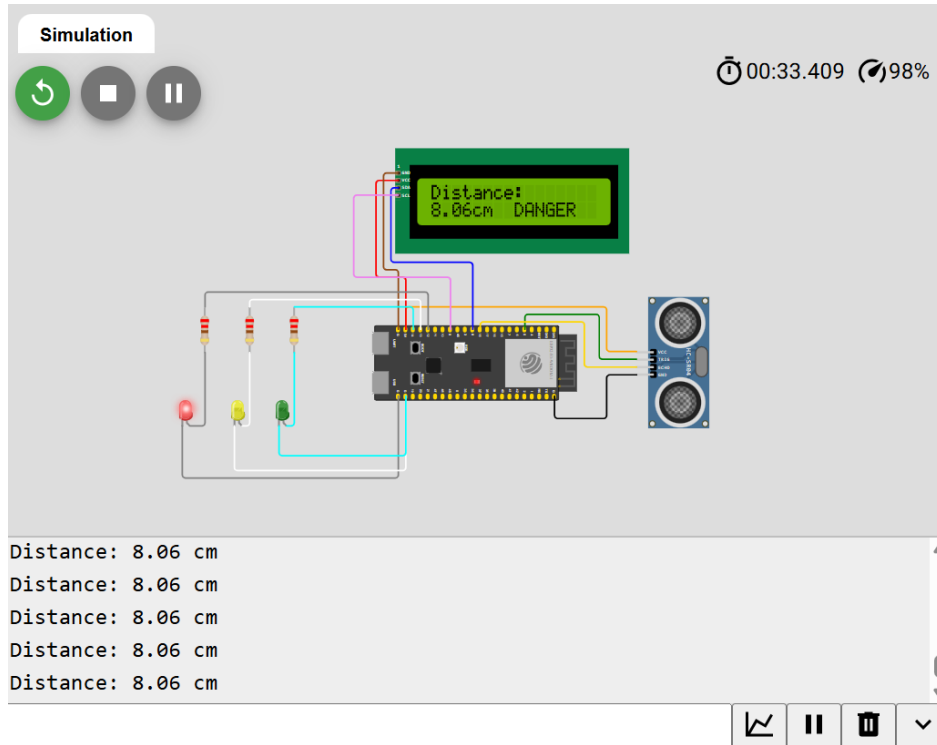


(a) The SAFE condition occurs when the object distance is greater than 50 cm. In this condition, the green LED turns on as a safe indicator.



(b) The WARNING condition occurs when the object distance is between 20 cm and 50 cm. In this condition, the yellow LED turns on as a alert indicator.

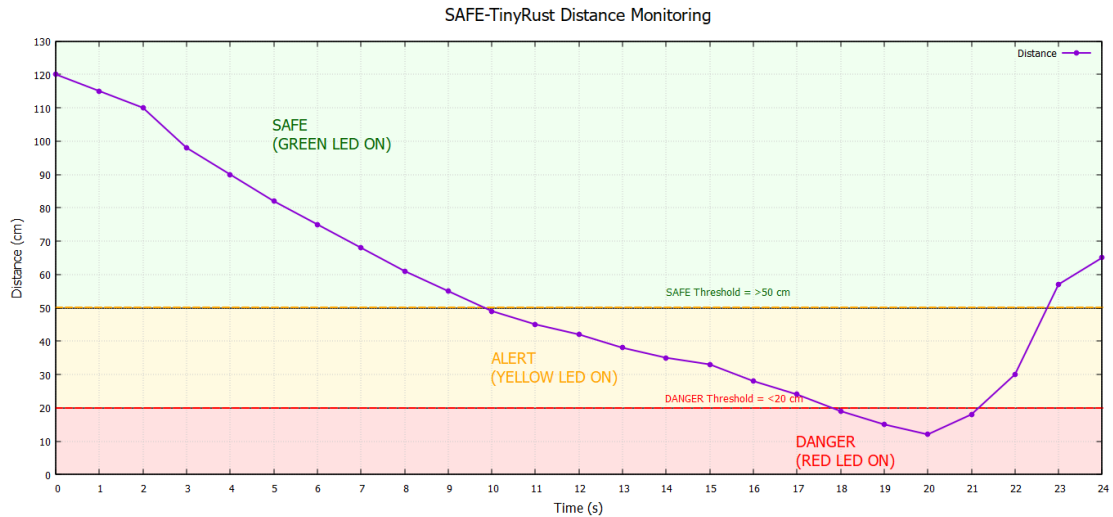
Gambar 3: Simulation results of the distance monitoring system.



(c) The DANGER condition occurs when the object distance is less than 20 cm. In this condition, the red LED turns on as a danger indicator.

Gambar 3: Simulation results of the distance monitoring system (continued).

Data and GNU Plot



Gambar 4: GNUPlot result of the SAFE-TinyRust distance monitoring system using HC-SR04 ultrasonic sensor on ESP32-S3.

GNU Plot Result Analysis

The simulation results demonstrate that the proposed SAFE-TinyRust system successfully performed adaptive real-time distance monitoring using the HC-SR04 ultrasonic

sensor integrated with the ESP32-S3 platform. Based on the GNUPlot graph, the measured distance gradually decreased from 120 cm to below the defined safety thresholds, allowing the system to classify environmental conditions into *SAFE*, *ALERT*, and *DANGER* states automatically.

When the measured distance was greater than 50 cm, the system operated in *SAFE* mode, indicated by the activation of the green LED. In this condition, the object was considered sufficiently far from the sensor, and no warning action was required. As the distance decreased into the range of 20–50 cm, the system transitioned into *ALERT* mode, where the yellow LED was activated as an early warning indicator. This adaptive transition demonstrates that the monitoring algorithm can respond dynamically to changing environmental conditions in real time.

Furthermore, when the detected distance dropped below 20 cm, the system entered the *DANGER* condition and activated the red LED warning indicator. This condition represents a critical proximity level requiring immediate attention. The graph also shows that the system could recover back to *ALERT* and *SAFE* conditions as the measured distance increased again, demonstrating stable sensor response and reliable threshold-based classification.

Overall, the simulation confirms that the SAFE-TinyRust framework can provide stable monitoring performance, fast response behavior, and clear safety-state visualization. The integration of Rust-based embedded programming with real-time sensor processing improves system reliability, memory safety, and low-latency operation, making the proposed method suitable for intelligent IoT monitoring and safety-critical embedded applications.

D. Method Advantages

Comparison with Reference Journal Methods

Tabel 2: Comparison of SAFE-TinyRust against methods from reference journals

Aspect	Methods from Reference Journals	SAFE-TinyRust (This Method)
Embedded Programming	Mostly uses C/C++ or conventional embedded programming	Uses Rust-based embedded programming with memory safety
TinyML Implementation	Focused only on lightweight inference	Integrates TinyML with adaptive monitoring and security layers
Fault Tolerance	Applied separately in safety-critical systems	Integrated directly into lightweight IoT monitoring architecture
Security Mechanism	Limited IoT communication security	Dynamic runtime security and communication validation
Memory Safety	Vulnerable to memory leaks and race conditions	Ownership-based memory safety using Rust
Real-Time Processing	Some systems still depend on cloud processing	Performs local inference directly on ESP32-S3
System Integration	Monitoring, AI, and security handled separately	Unified framework integrating AI, monitoring, safety, and IoT
Monitoring Capability	Mostly static monitoring systems	Adaptive real-time monitoring with SafetyScore mechanism
Actuator Response	Basic warning activation	Intelligent real-time LED warning response
Original Contribution	Focus on single-domain solutions	Combines TinyML, Rust, fault monitoring, and IoT security in one framework

Originality Contributions

Key contributions from the synthesis of FW1–FW15 that have not been explicitly addressed by prior researchers are summarized as follows:

- **Integration of Rust and TinyML for Embedded Safety Systems**

The proposed SAFE-TinyRust framework combines Rust-based memory-safe embedded programming with TinyML local inference on ESP32-S3. Previous studi-

es generally focused only on TinyML optimization or embedded software safety separately.

- **Adaptive SafetyScore Mechanism**

This work introduces an adaptive SafetyScore mechanism that combines distance monitoring results, fault monitoring status, and communication stability into one real-time decision-making parameter for intelligent warning systems.

- **Lightweight Fault Monitoring Architecture**

Unlike conventional fault tolerance systems that require complex hypervisor-based architectures, the proposed method implements lightweight runtime fault monitoring suitable for resource-constrained microcontrollers.

- **Dynamic Security Layer for IoT Embedded Systems**

The framework integrates dynamic security validation mechanisms including communication checking, runtime validation, and data integrity monitoring directly inside the embedded monitoring architecture.

- **Real-Time Local TinyML Inference**

The TinyML model performs inference directly on the ESP32-S3 without relying on cloud computing, improving latency, communication efficiency, and system responsiveness.

- **Unified Intelligent Embedded Monitoring Framework**

The originality of this research lies in combining TinyML, Rust safety mechanisms, adaptive monitoring, IoT communication, and intelligent actuator control into one unified lightweight embedded framework.

- **Safety-Critical IoT Monitoring Implementation**

The proposed framework is specifically designed for safety-critical IoT monitoring applications where reliability, security, low latency, and efficient resource utilization are essential.

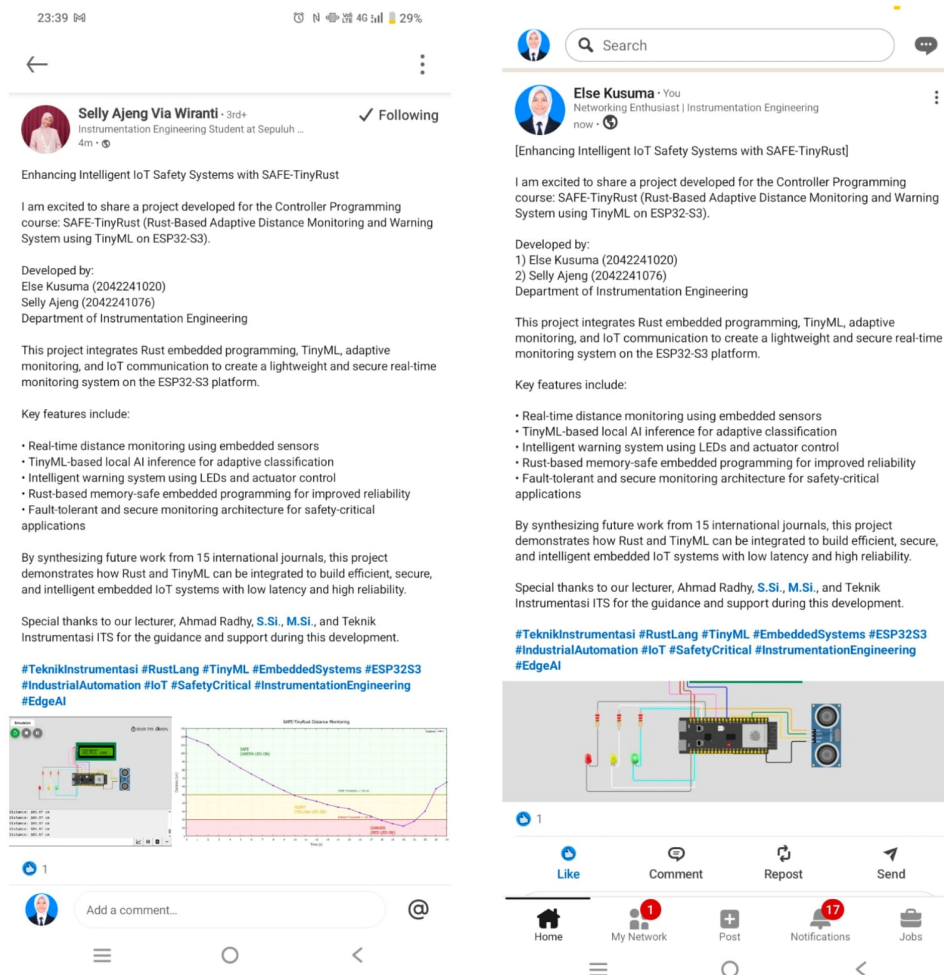
E. GitHub and LinkedIn

Link Repository GitHub:

- <https://github.com/elkuss/SAFE-TinyRust>
- <https://github.com/SellyAjeng/SAFE-TinyRust>

LinkedIn Links:

- [Else's LinkedIn Post](#)
- [Selly's LinkedIn Post](#)



Gambar 5: Screenshot of the LinkedIn publication showcasing the SAFE-TinyRust project uploaded by Else and Selly

Conclusion

Based on the systematic literature review of 15 reference journals, several important future work trends were identified, including TinyML optimization, Rust-based embedded programming, adaptive monitoring, fault tolerance, IoT security, and real-time intelligent systems. These future work directions were integrated into the development of the proposed SAFE-TinyRust framework as a unified intelligent embedded monitoring architecture for safety-critical IoT applications.

The proposed SAFE-TinyRust method combines Rust-based embedded programming, TinyML inference, adaptive monitoring, dynamic security mechanisms, and IoT communication on the ESP32-S3 platform. By utilizing Rust memory-safety features and lightweight local AI inference, the system is able to perform real-time distance monitoring with low latency, efficient resource utilization, and improved operational reliability.

The simulation results demonstrated that the system successfully classified environmental conditions into *SAFE*, *ALERT*, and *DANGER* states based on ultrasonic sensor measurements. The monitoring system showed stable real-time performance, adaptive response behavior, and reliable actuator activation through LED indicators according to predefined distance thresholds. The GNUPlot analysis also confirmed that the framework could dynamically transition between safety states while maintaining consistent monitoring operation.

Overall, the SAFE-TinyRust framework offers several advantages compared to previous methods, including memory-safe embedded programming using Rust, local TinyML inference without cloud dependency, lightweight fault monitoring, adaptive *SafetyScore* mechanisms, and integrated IoT security validation. The proposed framework has strong potential for future development in broader intelligent monitoring applications, such as industrial safety systems, smart transportation, healthcare monitoring, and other safety-critical embedded IoT environments. Further research may focus on implementing more advanced TinyML models, improving sensor accuracy, integrating additional communication protocols, and performing real-world hardware validation.

References

- [1] S. Yu and Z. Wang, “An Empirical Study on Bugs in Rust Programming Language,” *Journal of Systems and Software*, vol. 214, pp. 112034, 2024.
- [2] Zhu et al., “Learning and Programming Challenges of Rust: A Mixed-Methods Study,” *Empirical Software Engineering*, vol. 27, no. 5, pp. 1–27, 2022.
- [3] R. Immonen and T. Hämäläinen, “Tiny Machine Learning for Resource-Constrained Microcontrollers,” *Journal of Sensors*, vol. 2022, Article ID 7437023, pp. 1–11, 2022.
- [4] L. Angione et al., “A System-Level Test Methodology for Communication Peripherals in System-on-Chips,” *Microelectronics Reliability*, vol. 162, pp. 115678, 2025.
- [5] Saha et al., “Machine Learning for Microcontroller-Class Hardware: A Review,” *IEEE Access*, vol. 10, pp. 123456–123478, 2022.
- [6] G. Roger L and P. Nisha, “Fleet Management System for Electric Buses using ESP32 Microcontroller,” *International Journal of Intelligent Transportation Systems*, vol. 8, no. 2, pp. 55–63, 2024.
- [7] M. Xie et al., “Basic Developing Environment of Microcontroller-based Monitoring System for Physiological Signals,” *Biomedical Signal Processing and Control*, vol. 75, pp. 103547, 2022.
- [8] C. Axiotidis, E. Konstantopoulou, and N. Slavkos, “A Wireless Sensor Network IoT Platform for Consumption and Quality Monitoring of Water,” *Sensors*, vol. 25, no. 3, pp. 1120–1136, 2025.
- [9] S. Lozano, J. Fernández, and J. Carretero, “Applying Hypervisor-Based Fault Tolerance Techniques to Safety-Critical Embedded Systems,” *Future Generation Computer Systems*, vol. 158, pp. 215–228, 2026.
- [10] T. Malche, M. Joshi, G. M. Upadhyay, and P. K. Soni, “Automated Tomato Leaf Disease Detection and Alert System using Internet of Things and TinyML,” *Computers and Electronics in Agriculture*, vol. 225, pp. 109876, 2026.
- [11] A. D. Gupta et al., “Design and Development of Test Set-up with Integrated Digital Controller for Temperature Radiation of Materials,” *Measurement*, vol. 238, pp. 115432, 2026.

- [12] C. Baron and V. Louis, “Framework and Tooling Proposals for Agile Certification of Safety-Critical Embedded Software in Avionics Systems,” *Software Quality Journal*, vol. 31, no. 4, pp. 889–912, 2023.
- [13] Y. B. Choi, P. C. Hong, and Y. S. Park, “Layered Control Architectures for AI Safety: A Cybersecurity-Oriented Systems Framework,” *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 1, pp. 44–58, 2026.
- [14] S. Varoumas, B. Pesin, B. Vaugon, and E. Chailloux, “Programming Microcontrollers through High-Level Abstractions: The OMicroB Project,” *Journal of Computer Languages*, vol. 77, pp. 101228, 2023.
- [15] N. Moghadasi, Z. A. Collier, D. C. Loose, M. C. Gunn, M. E. Gunn, I. Linkov, K. R. Jackson, and J. H. Lambert, “Safety and Security Priorities of Smart Embedded Devices in a Technology Enterprise,” *Reliability Engineering and System Safety*, vol. 272, pp. 112657, 2026.